CSCI 6461 | WEEK 1

Introduction to Computing Systems & Binary Foundations

Abstraction, representation, and quantitative performance

Dr. J. Burns

Computer Systems Architecture

What Computer Architecture Studies

- Computer architecture is not just "parts of a computer." It studies the **interface between software behavior and hardware implementation**.
- The central question is: **what must the machine promise to software, and how can hardware deliver that promise efficiently?**
- Architecture includes instruction sets, processor organization, memory systems, and performance trade-offs.
- A correct machine is not automatically a good machine; architecture also asks whether it is fast, efficient, scalable, and practical.

Course Focus

We will use ARM AArch64 as the concrete RISC architecture, but Week 1 builds the representation and performance foundations first.

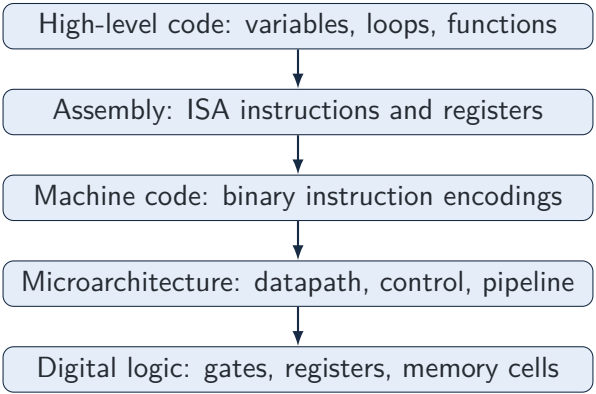
The Core Abstraction Problem

- Programmers want to write algorithms, not manipulate voltage levels.
- Hardware only stores and transforms bit patterns.
- Abstraction layers make this gap manageable.
- Each layer hides detail, but hidden detail can still affect performance.

Important tension

Abstraction makes systems usable. Architecture explains when abstraction leaks: performance, memory layout, integer overflow, and instruction costs.

System Layers: From Program to Gates



- Architecture connects these layers rather than treating them as unrelated topics.

A Small Example Across the Layers

High-level intent

```
c = a + b;
```

- If `a` and `b` are already in registers, a compiler may choose an addition instruction.
- In AArch64, this could be conceptually written as

```
ADD X0, X1, X2
```

- The instruction means: **X0 gets X1 + X2**.

- The assembler encodes the instruction into 32 bits.
- Hardware decodes fields for operation and register numbers.
- The ALU performs the addition and writes the result.

ISA vs. Microarchitecture

ISA: what software sees

- Instructions
- Registers
- Addressing behavior
- Architectural state
- Defined results

Microarchitecture: how hardware does it

- Datapath
- Control logic
- Pipeline stages
- Caches
- Execution units

Key idea: Two processors may run the same AArch64 program correctly but have very different performance.

The Stored-Program Model

- Instructions and data are both stored in memory as bit patterns.
- A program counter identifies the next instruction to fetch.
- The processor repeatedly performs fetch, decode, execute, and state update.
- Branches and calls modify the normal next-instruction sequence.
- This model is simple, but it is the conceptual base for pipelines and caches.

Why this matters later

If instructions live in memory, then memory speed affects not only data access but also instruction delivery.

Core Hardware Components

- **CPU:** executes instructions.
- **Registers:** hold nearby operands and results.
- **Main memory:** holds active programs and data.
- **Storage:** keeps persistent data.
- **I/O:** connects the machine to the outside world.

Architectural consequence

The processor is fast, memory is slower, and storage is much slower. Performance depends heavily on movement across this hierarchy.

Why Binary Is Enough

- Digital circuits reliably distinguish two voltage ranges, represented as 0 and 1.
- A single bit carries little information; groups of bits carry many possibilities.
- With n bits, there are 2^n distinct patterns.
- Those patterns can represent numbers, characters, instructions, addresses, or control fields.
- The hardware does not know the "meaning" of a pattern by itself.

Core warning

Bits are representation. Meaning comes from interpretation.

Unsigned Binary: Place Value

Bit position	5	4	3	2	1	0
Weight	32	16	8	4	2	1
Bits	1	0	1	1	0	1

- Binary is positional just like decimal, but the weights are powers of two.
- A 1 contributes its weight; a 0 contributes nothing.
- `101101` therefore represents

$$32 + 8 + 4 + 1 = 45$$

- With fixed-width hardware, the number of available positions is limited.

Unsigned Range and Fixed Width

- An n-bit unsigned value has 2^n possible patterns.
- The smallest value is 0.
- The largest value is

$$2^n - 1$$

- Four unsigned bits represent 0 through 15.
- Eight unsigned bits represent 0 through 255.
- Fixed width is not a formatting detail; it determines what values are representable.

Architecture connection

Register widths, immediate fields, and address fields all impose representational limits.

Converting Decimal to Binary

Example: convert 45 to binary

$$45 = 32 + 8 + 4 + 1$$

Weight	32	16	8	4	2	1
Use it?	1	0	1	1	0	1

- Select the powers of two that sum to the decimal value.
- Write 1 where a weight is used and 0 where it is not.
- Therefore

$$45_{10} = 101101_2$$

Converting Binary to Decimal

Example: convert 110101

$$110101_2$$

Weight	32	16	8	4	2	1
Bits	1	1	0	1	0	1

$$32 + 16 + 4 + 1 = 53$$

Result

$$110101_2 = 53_{10}$$

Why Hexadecimal Exists

- Long binary strings are difficult for humans to read.
- Hexadecimal uses 16 symbols:

$$0, 1, \ldots, 9, A, B, C, D, E, F$$

- One hexadecimal digit represents exactly four bits.
- Two hexadecimal digits represent one byte.
- Hexadecimal therefore compresses binary without hiding the underlying bit boundaries.
- Addresses and instruction encodings are commonly written in hexadecimal.

Binary and Hexadecimal

Binary	Hex	Binary	Hex
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

Example

$$0011\ 1101_2 = \texttt{0x3D}$$

The Signed-Integer Problem

- Unsigned binary uses all available patterns for zero and positive values.
- Real programs also require negative integers.
- We therefore need a convention that assigns some bit patterns to negative values.
- A good representation should make arithmetic hardware simple.
- It should also provide predictable fixed-width behavior.
- Modern processors overwhelmingly use **two's-complement representation**.

Why Not Sign-Magnitude?

- A simple idea is to reserve one bit for the sign.
- The remaining bits store the magnitude.
- This is easy for humans to understand.
- But it creates both $+0$ and -0.

Hardware problem

Addition becomes conditional: the arithmetic hardware must consider signs and magnitudes separately.

- This complicates datapath design.
- Two's complement avoids much of this complexity.

Two's-Complement Range

- In n bits, two's complement still has 2^n total patterns.
- The lower half of the number line is negative.
- The upper half includes zero and positive values.
- The range is

$$-2^{n-1} \quad \text{through} \quad 2^{n-1} - 1$$

- Eight bits represent

$$-128 \quad \text{through} \quad +127$$

- The range is asymmetric because zero uses one nonnegative pattern.

Creating a Negative Value

Rule for forming $-x$ in two's complement

Write $+x$ in the required width, invert all bits, then add 1.

$+5$ in 8 bits:	00000101
Invert bits:	11111010
Add 1:	11111011

- Therefore 11111011 represents -5 in 8-bit two's complement.
- The width is part of the representation; do not drop leading bits casually.

Interpreting a Negative Pattern

Example: interpret `11110110` as an 8-bit signed integer

Original:	`11110110`
Invert:	`00001001`
Add 1:	`00001010` $= 10$
Result:	-10

- A leading 1 signals a negative value in two's complement.
- The invert-and-add-one procedure finds the magnitude.

Same Bits, Different Interpretation

Bit pattern	Interpretation	Value
`11111111`	Unsigned 8-bit	255
`11111111`	Signed 8-bit two's complement	-1
`11111111`	Byte of raw data	Depends on format

- The bits did not change; only the interpretation changed.
- Low-level bugs often come from using the wrong interpretation.
- Architecture forces us to track width and signedness explicitly.

Binary Addition and Subtraction

Addition

$$0101_2 + 0011_2 = 1000_2$$

- Binary addition uses carries just like decimal addition.

Subtraction

$$A - B = A + (-B)$$

- Two's complement lets the same adder support subtraction.
- This is a representation choice with hardware consequences.

Carry vs. Signed Overflow

- Carry-out matters for unsigned arithmetic.
- Signed overflow is different: it means the signed result is outside the representable range.
- In 4-bit signed arithmetic, the maximum positive value is $+7$.
- 0111 represents $+7$ and 0011 represents $+3$.
- $$0111 + 0011 = 1010$$

  but $+10$ cannot fit in four signed bits.
- The negative-looking result signals signed overflow, not a valid answer of -6.

Sign Extension and Truncation

- Sign extension widens a signed value without changing its mathematical value.
- It repeats the original most significant bit into the new high-order bits.
- Eight-bit -5 is

$$\texttt{11111011}$$

- Sixteen-bit -5 is

$$\texttt{11111111 11111011}$$

- Truncation does the opposite: it discards high-order bits.
- Truncation can destroy information and change the value completely.

Representation Mistakes That Matter

- Treating signed data as unsigned can turn -1 into a large positive value.
- Forgetting width can make a conversion appear correct but mean the wrong thing.
- Ignoring overflow can produce plausible-looking but incorrect results.
- Misreading hexadecimal can hide the bit fields inside an instruction or address.
- Confusing mathematical integers with fixed-width machine integers causes subtle bugs.
- These are not just programming mistakes; they are architecture-level interpretation mistakes.

What Does CPU Performance Mean?

- For the same workload, less execution time means higher performance.
- Elapsed time may include I/O, operating-system activity, and waiting.
- CPU time focuses on processor time spent executing the program.
- Clock rate is visible and marketable, but incomplete.
- A faster clock can be offset by more instructions or more cycles per instruction.
- Architecture uses equations so performance comparisons do not rely on slogans.

Latency vs. Throughput

- **Latency** is the time for one task to complete.
- **Throughput** is the amount of work completed per unit time.
- A processor improvement may reduce latency, increase throughput, or both.
- Pipelining often improves throughput more directly than the latency of one instruction.
- Users often care about latency; servers often care heavily about throughput.
- Distinguishing these terms prevents confusion when evaluating architectural techniques.

Clock Rate and Cycle Time

- Clock rate is the number of cycles per second.
- Cycle time is the duration of one cycle.
- They are reciprocals:

$$\text{Cycle Time} = \frac{1}{\text{Clock Rate}}$$

- A 2 GHz processor has a 0.5 ns cycle time.
- A 3 GHz processor has a shorter cycle time, but may not finish a program sooner.
- Program time also depends on instruction count and CPI.

Instruction Count

- Instruction count is the number of **dynamic machine instructions** executed.
- It is not just the number of lines in a program.
- Loops can execute the same instruction many times.
- Instruction count depends on the algorithm, compiler, ISA, and input data.
- A program compiled for two different ISAs may execute different numbers of instructions.
- Reducing instruction count helps only if CPI and clock rate do not worsen enough to cancel the gain.

Cycles Per Instruction (CPI)

- CPI is the average number of clock cycles per executed instruction.
- A CPI of 1 does not mean every instruction takes exactly one cycle.
- It is an average across the dynamic instruction stream.
- Memory stalls, pipeline hazards, and branch behavior can increase CPI.
- Better microarchitecture often tries to reduce CPI without changing the ISA.
- CPI is where implementation details often reappear despite software abstraction.

The Iron Law of Processor Performance

The Iron Law

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$\text{CPU Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

- Instruction count connects to software, compiler, and ISA.
- CPI connects to microarchitecture and memory behavior.
- Clock cycle time connects to circuit timing and design complexity.

Iron Law Worked Example

Processor A

$$IC = 1.0 \times 10^9, \qquad CPI = 2.0, \qquad Clock = 2.0\ GHz$$

$$Time = \frac{1.0 \times 10^9 \times 2.0}{2.0 \times 10^9} = 1.0\ s$$

Processor B

$$IC = 1.2 \times 10^9, \qquad CPI = 1.2, \qquad Clock = 2.4\ GHz$$

$$Time = \frac{1.2 \times 10^9 \times 1.2}{2.4 \times 10^9} = 0.6\ s$$

Why GHz Comparisons Fail

- Processor B in the example executes more instructions than Processor A.
- It still wins because its CPI and clock rate more than compensate.
- A higher clock can be defeated by poor CPI.
- A lower instruction count can be defeated by slower cycles.
- A design that improves one term may worsen another term.
- The Iron Law forces the full trade-off to be made explicit.

Speedup

Definition

$$\text{Speedup} = \frac{\text{Old Execution Time}}{\text{New Execution Time}}$$

- If time falls from 10 seconds to 5 seconds, speedup is $2\times$.
- If time falls from 10 seconds to 8 seconds, speedup is $1.25\times$.
- Speedup greater than 1 means improvement.
- Speedup below 1 means slowdown.
- Always specify the workload and the baseline system.

Week 1 Synthesis and Transition

- Abstraction explains how high-level programs can execute on physical hardware.
- The ISA is the key boundary between machine-level software and hardware implementation.
- Binary and hexadecimal let us read the representations used for data and instructions.
- Two's complement shows how representation choices simplify arithmetic hardware.
- The Iron Law gives a disciplined way to reason about processor performance.
- Next, we apply these foundations to the AArch64 programmer's model and basic assembly instructions.